

codigo prueba

database.py

```
import os
from dotenv import load_dotenv
from supabase import create_client, Client

# Cargar las variables del archivo .env
load_dotenv()

SUPABASE_URL = os.getenv("supabase_url")
SUPABASE_KEY = os.getenv("supabase_key")

def obtener_conexion() -> Client:
    """
    Crea e inicializa el cliente de Supabase utilizando las variables de entorno.
    """
    if not SUPABASE_URL or not SUPABASE_KEY:
        raise ValueError("Error: Faltan las variables SUPABASE_URL o SUPABASE_KEY en el archivo .env")

    return create_client(SUPABASE_URL, SUPABASE_KEY)
```

app_controller.py

```
from models.product_dao import ProductDAO
from models.empleado_dao import EmpleadoDAO
from views.consola_view import ConsolaView

class AppController:
    """Capa Controlador: Procesa entradas del usuario e invoca DAOs[cite: 2, 3]."""

    def __init__(self):
        self.view = ConsolaView()

    def ejecutar(self):
        while True:
            opcion = self.view.menu_principal()
            if opcion == '1':
                self.gestionar_productos()
            elif opcion == '2':
                self.gestionar_empleados()
            elif opcion == '3':
                self.view.mostrar_mensaje("Finalizando la aplicaci3n...")
                break
            else:
                self.view.mostrar_mensaje("Opci3n no v3lida. Intente de nuevo.")

    def gestionar_productos(self):
        while True:
            sub_opc = self.view.menu_crud("Productos")
            try:
                if sub_opc == '1':
                    c, n, d, p = self.view.captura_producto()
                    ProductDAO.crear(c, n, d, p)
                    self.view.mostrar_mensaje("Producto creado exitosamente.")
                elif sub_opc == '2':
                    prods = ProductDAO.obtener_todos()
                    self.view.mostrar_productos(prods)
                elif sub_opc == '3':
                    id_p, n, p = self.view.capturar_actualizacion_producto()
                    ProductDAO.actualizar(id_p, n, p)
                    self.view.mostrar_mensaje("Producto actualizado exitosamente.")
                elif sub_opc == '4':
                    id_p = self.view.solicitar_id("producto")
                    ProductDAO.eliminar(id_p)
                    self.view.mostrar_mensaje("Producto eliminado exitosamente.")
                elif sub_opc == '5':
                    break
            except Exception as e:
                self.view.mostrar_mensaje(f"Error procesando solicitud: {e}")

    def gestionar_empleados(self):
        while True:
            sub_opc = self.view.menu_crud("Empleados")
            try:
                if sub_opc == '1':
                    n, a, d, r, e = self.view.capturar_empleado()
```

```

        EmpleadoDAO.crear(n, a, d, r, e)
        self.view.mostrar_mensaje("Empleado registrado exitosamente.")
    elif sub_opc == '2':
        emps = EmpleadoDAO.obtener_todos()
        self.view.mostrar_empleados(emps)
    elif sub_opc == '3':
        id_e, r, e_mail = self.view.capturar_actualizacion_empleado()
        EmpleadoDAO.actualizar(id_e, r, e_mail)
        self.view.mostrar_mensaje("Empleado actualizado exitosamente.")
    elif sub_opc == '4':
        id_e = self.view.solicitar_id("empleado")
        EmpleadoDAO.eliminar(id_e)
        self.view.mostrar_mensaje("Empleado eliminado exitosamente.")
    elif sub_opc == '5':
        break
except Exception as e:
    self.view.mostrar_mensaje(f"Error procesando solicitud: {e}")

```

empleado_dao.py

```
from config.database import obtener_conexion
```

```

class EmpleadoDAO:
    """
    PatrÃ³n DAO: Encapsula la persistencia y operaciones CRUD para Empleados[cite: 2].
    """

    @staticmethod
    def crear(nombre: str, apellido: str, documento_identidad: str, rol: str, email: str) -> dict:
        """Registra un nuevo empleado en la base de datos[cite: 3]."""
        supabase = obtener_conexion()
        datos = {
            "nombre": nombre,
            "apellido": apellido,
            "documento_identidad": documento_identidad,
            "rol": rol,
            "email": email
        }
        respuesta = supabase.table("empleado").insert(datos).execute()
        return respuesta.data

    @staticmethod
    def obtener_todos() -> list:
        """Obtiene el listado completo de empleados[cite: 3]."""
        supabase = obtener_conexion()
        respuesta = supabase.table("empleado").select("*").execute()
        return respuesta.data

    @staticmethod
    def actualizar(id_empleado: str, rol: str = None, email: str = None) -> dict:
        """Actualiza el rol o email de un empleado[cite: 3]."""
        supabase = obtener_conexion()
        datos = {}
        if rol:
            datos["rol"] = rol
        if email:
            datos["email"] = email

        respuesta = supabase.table("empleado").update(datos).eq("id", id_empleado).execute()
        return respuesta.data

    @staticmethod
    def eliminar(id_empleado: str) -> dict:
        """Elimina un empleado de la base de datos mediante su ID[cite: 3]."""
        supabase = obtener_conexion()
        respuesta = supabase.table("empleado").delete().eq("id", id_empleado).execute()
        return respuesta.data

```

product_dao.py

```
from config.database import obtener_conexion
```

```

class ProductDAO:
    """
    Se a crean las operaciones CRUD en la tabla producto
    """

    @staticmethod
    def crear(codigo_barras:str, nombre: str, descripcion:str, precio_venta:float) -> dict:

```

```

"""
    Crea un nuevo registro en la tabla producto
"""

supabase= obtener_conexion()
info={
    "codigo_barras":codigo_barras,
    "nombre":nombre,
    "descripcion":descripcion,
    "precio_venta":precio_venta
}

respuesta =supabase.table("producto").insert(info).execute()
return respuesta.data

@staticmethod
def obtener_todos() -> list:
    """
        Lee la tabla y trae todos los registros de la tabla
    """

    supabase = obtener_conexion()
    respuesta = supabase.table("producto").select("*").execute()
    return respuesta.data

@staticmethod
def actualizar(id_producto: str, nombre: str = None, precio_venta: float=None) -> dict:
    """
        Actualiza los campos especificos de un producto ya existente en la tabla
    """

    supabase = obtener_conexion()
    datos_actualizar = {}
    if nombre:
        datos_actualizar["nombre"] = nombre
    if precio_venta:
        datos_actualizar["precio_venta"] = precio_venta

    respuesta = supabase.table("producto").update(datos_actualizar).eq("id", id_producto).execute()
    return respuesta.data

@staticmethod
def eliminar(id_producto: str)->dict:
    """
        Elimina un registro de producto por su ID
    """

    supabase = obtener_conexion()
    respuesta = supabase.table("producto").delete().eq("id", id_producto).execute()
    return respuesta.data

```

consola_view.py

```

class ConsolaView:
    """
        Capa de vista dedicada para manejar la interfaz de usuario en la terminal o consola
    """

    @staticmethod
    def menu_principal()-> str:
        print("\n" + "="*35)
        print("    ARQUITECTURA DE SOFTWARE")
        print("    Actividad evaluativa eje 2")
        print("="*35)
        print("1. Gestión de Productos")
        print("2. Gestión de Empleados")
        print("3. Salir")
        return input("Seleccione una opción: ")

    @staticmethod
    def menu_crud(entidad: str) -> str:
        print(f"\n--- MENÚ DE {entidad.upper()} ---")
        print("1. Crear / Registrar")
        print("2. Listar Todos")
        print("3. Actualizar")
        print("4. Eliminar")
        print("5. Volver al menú principal")
        return input("Seleccione una opción: ")

    @staticmethod

```

```

def captura_producto() -> tuple:
    print("\n -- Registro de producto--" )
    codigo = input("Código de barras: ")
    nombre = input("Nombre del producto: ")
    desc = input("Descripción: ")
    precio = float(input("Precio de venta: "))
    return codigo, nombre, desc, precio

@staticmethod
def capturar_actualizacion_producto() -> tuple:
    id_prod = input("\nIngrese el UUID del producto a actualizar: ")
    nombre = input("Nuevo nombre (dejar vacío para no cambiar): ")
    precio_str = input("Nuevo precio (dejar vacío para no cambiar): ")
    precio = float(precio_str) if precio_str else None
    return id_prod, nombre, precio

# --- ENTRADAS DE EMPLEADOS ---
@staticmethod
def capturar_empleado() -> tuple:
    print("\n-- Registro de Empleado --")
    nombre = input("Nombre: ")
    apellido = input("Apellido: ")
    doc = input("Documento de identidad: ")
    rol = input("Rol (ej. Cajero, Administrador): ")
    email = input("Correo electrónico: ")
    return nombre, apellido, doc, rol, email

@staticmethod
def capturar_actualizacion_empleado() -> tuple:
    id_emp = input("\nIngrese el UUID del empleado a actualizar: ")
    rol = input("Nuevo rol (dejar vacío para no cambiar): ")
    email = input("Nuevo email (dejar vacío para no cambiar): ")
    return id_emp, rol, email

# --- SALIDAS Y TABLAS ---
@staticmethod
def solicitar_id(entidad: str) -> str:
    return input(f"\nIngrese el UUID del {entidad} a eliminar: ")

@staticmethod
def mostrar_productos(productos: list):
    print("\n" + "-"*65)
    print(f"{'ID (UUID)':<38} | {'NOMBRE':<15} | {'PRECIO':<8}")
    print("-" * 65)
    if not productos:
        print("No hay productos registrados.")
    for p in productos:
        print(f"{p.get('id'):<38} | {p.get('nombre'):<15} | ${p.get('precio_venta'):<8}")
    print("-" * 65)

@staticmethod
def mostrar_empleados(empleados: list):
    print("\n" + "-"*75)
    print(f"{'ID (UUID)':<38} | {'NOMBRE COMPLETO':<20} | {'ROL':<10}")
    print("-" * 75)
    if not empleados:
        print("No hay empleados registrados.")
    for e in empleados:
        nombre_completo = f"{e.get('nombre')} {e.get('apellido')}"
        print(f"{e.get('id'):<38} | {nombre_completo:<20} | {e.get('rol'):<10}")
    print("-" * 75)

@staticmethod
def mostrar_mensaje(mensaje: str):
    print(f"\n[SISTEMA]: {mensaje}")

```